SOLID Class Exercise
Krishank Kureti
PES2UG23CS285
Sem 6 ‘E’

SOLID Principles Justification

1. Single Responsibility Principle (SRP) – `SRP_StudentManagementSystem.java`

The initial `StudentBad` class mixed data storage, GPA calculation, report generation, and email sending—multiple reasons to change. The refactored version separates these into `Student` (data), `GradeCalculator` (GPA), `ReportGenerator` (formatting), and `NotificationService` (communication). Each class now has a single, well-defined responsibility, making the system easier to maintain and test.

2. Open/Closed Principle (OCP) – `OCP_DiscountCalculator.java`

The `DiscountCalculatorBad` used conditional statements for each product type, requiring modification to add new discounts. The refactored code defines an abstraction `DiscountStrategy` with concrete implementations like `ElectronicsDiscount`. The `DiscountCalculator` registers strategies and delegates calculation, allowing new discount types to be added without modifying existing code—open for extension, closed for modification.

3. Liskov Substitution Principle (LSP) – `LSP_BankingSystem.java`

In the bad version, `FixedDepositAccountBad` extended `AccountBad` but overrode `withdraw()` to throw an exception under certain conditions, breaking substitutability. The refactored design introduces an abstract `Account` with clear contracts for withdrawable accounts. Fixed deposits, which have different withdrawal behavior, are modeled as a separate class `FixedDepositAccount`, ensuring that subtypes of `Account` can be substituted without unexpected behavior.

4. Interface Segregation Principle (ISP) – `ISP_SmartHomeSystem.java`

The monolithic `SmartDeviceBad` interface forced all devices to implement irrelevant methods (e.g., `setTemperature` for a light). The refactored version splits into focused interfaces: `Switchable`, `TemperatureControl`, `AudioPlayback`, and `BrightnessControl`. Devices implement only the interfaces they need, avoiding dummy methods and making the system more modular and client-friendly.

5. Dependency Inversion Principle (DIP) – `DIP_NotificationSystem.java`

High-level services like `NotificationService` depend directly on the `NotificationChannel` abstraction rather than concrete classes like `EmailService`. Concrete channels implement the abstraction, and dependencies are injected via the constructor. This inverts the dependency, allowing new channels (e.g., WhatsApp) to be added without modifying any high-level code, and facilitates testing and flexibility.